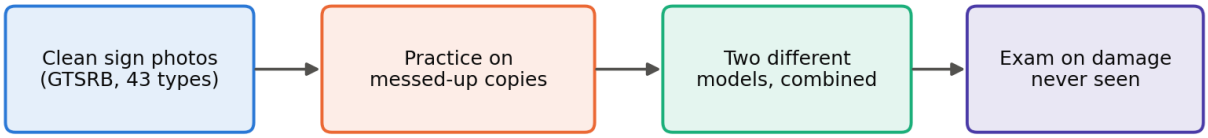
# RoadLens, explained simply

What the project does, how it does it, and what happens when you press *Run all* in the Colab notebook **RoadLens_standalone.ipynb**.

## The problem

Self-driving cars must read traffic signs. On clean photos computers already get about 99 out of 100 right. But real cameras see rain, fog, snow, darkness, shaking and grainy pictures, and then the same computer suddenly makes many mistakes. RoadLens builds a sign reader that keeps working when the picture is damaged, and measures honestly how well it does.

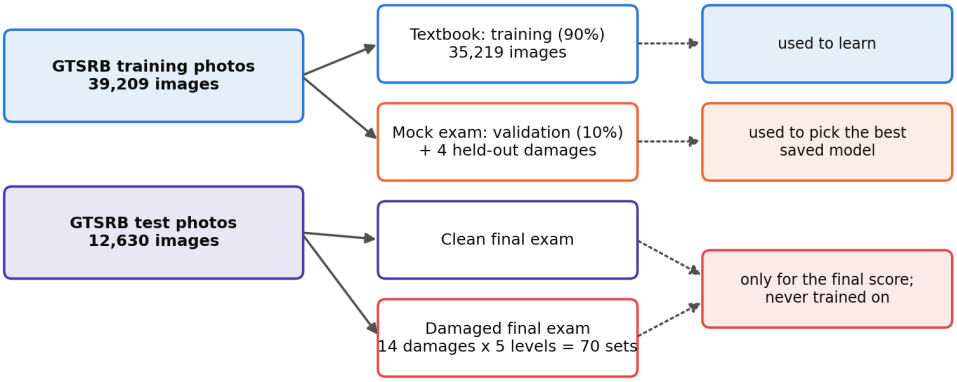
## The idea in one picture




*Goal: a sign reader that stays correct in rain, fog, snow, blur, noise and bad light*

Think of a student: learn from a **textbook** (clean photos), do **practice questions** (changed copies), take a **mock exam** to decide when you are ready, and finally a **surprise exam** with question types you never practised (the 14 kinds of damage).

# Step 1 - the data and the exams




Photos of the same physical sign are kept on one side, so the mock exam has no near-copies of training photos. Belgian signs (BTSD) are a second, 'other country' exam.

• Every photo is shrunk to **64 x 64 pixels**, like a sign seen from far away. Damage is added after shrinking.

• The **final exam** is the test set with 14 kinds of damage, each at 5 strengths (S1 light ... S5 heavy): noise (grain), blur (shaking, zoom, out of focus), weather (snow, fog, frost, too bright), and digital damage (low contrast, blocky pixels, wobbly shapes, JPEG compression).

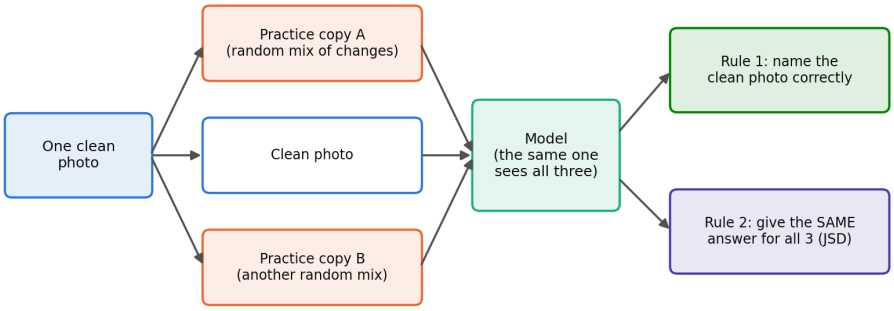
• The model **never** practises on these 14. That is what makes the exam fair.

• The damage strengths were made stronger on 15 Oct 2026 so that S5 is really hard; they are listed in CORRUPTIONS.md and in RoadLens_simple_guide.pdf.

## Step 2 - the models

| Model | In simple words | Role |
|---|---|---|
| SmallCNN | a small network trained from zero | the 'ruler': other scores are compared to it (mCE = 100) |
| ResNet-18 | a well-known picture network that already learned from 1 million everyday photos | strong baseline |
| DeiT-S | a 'transformer' that cuts the picture into small squares and looks at how they relate | a different way of seeing |
| Fusion | ResNet-18 + DeiT-S together, with a gate that decides whom to trust | the proposed model |

# Step 3 - how the models practise



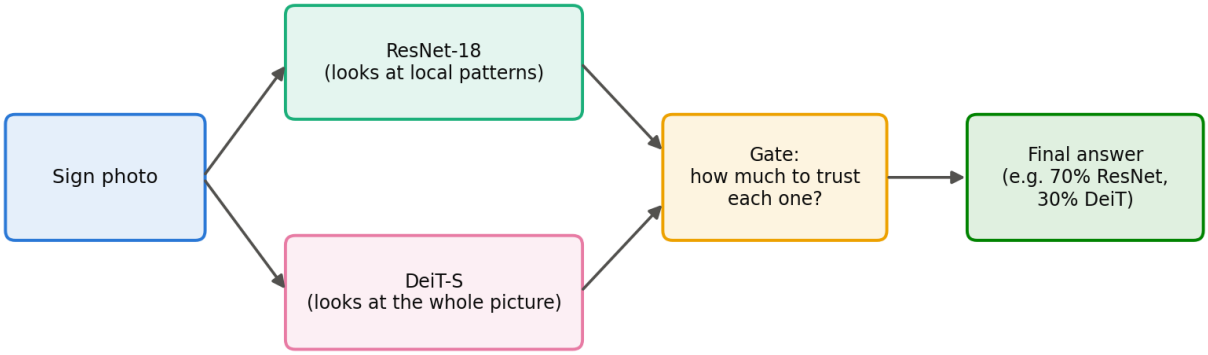

Changes used: contrast stretch, colour levels, rotate, shear, shift, plus waves, random sharpen/soften, gentle bends, colour curves. None of them is one of the 14 exam damages.

• **Baseline training**: only clean photos with tiny colour changes. No left-right flipping, because a 'keep left' sign would turn into 'keep right'.

• **RoadLens training**: every photo also gets two practice copies made by mixing random changes (AugMix and PRIME style). The model must name the clean photo correctly *and* give the same answer for all three versions. This teaches it to ignore the kind of change rather than memorise one look.

• Training stops early when the score on the mock exam stops improving, and the best version is kept.

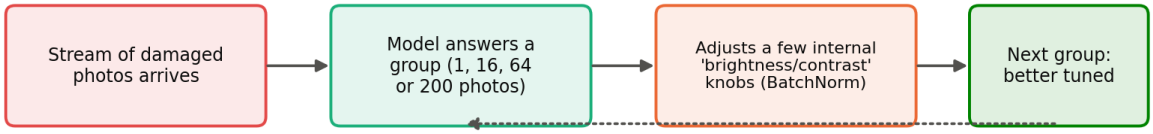
## Step 4 - two heads are better than one




Fair comparisons kept: a plain 50/50 average, and 'trust whichever is more confident'.

The two models make different mistakes, so combining them helps. The gate looks at each photo and how sure each model is, then mixes their answers.

# Step 5 - adapting while driving




TENT = make the model more sure of itself on the new conditions; no labels needed. With 1 photo at a time it has too little to learn from.

## Step 6 - how success is measured

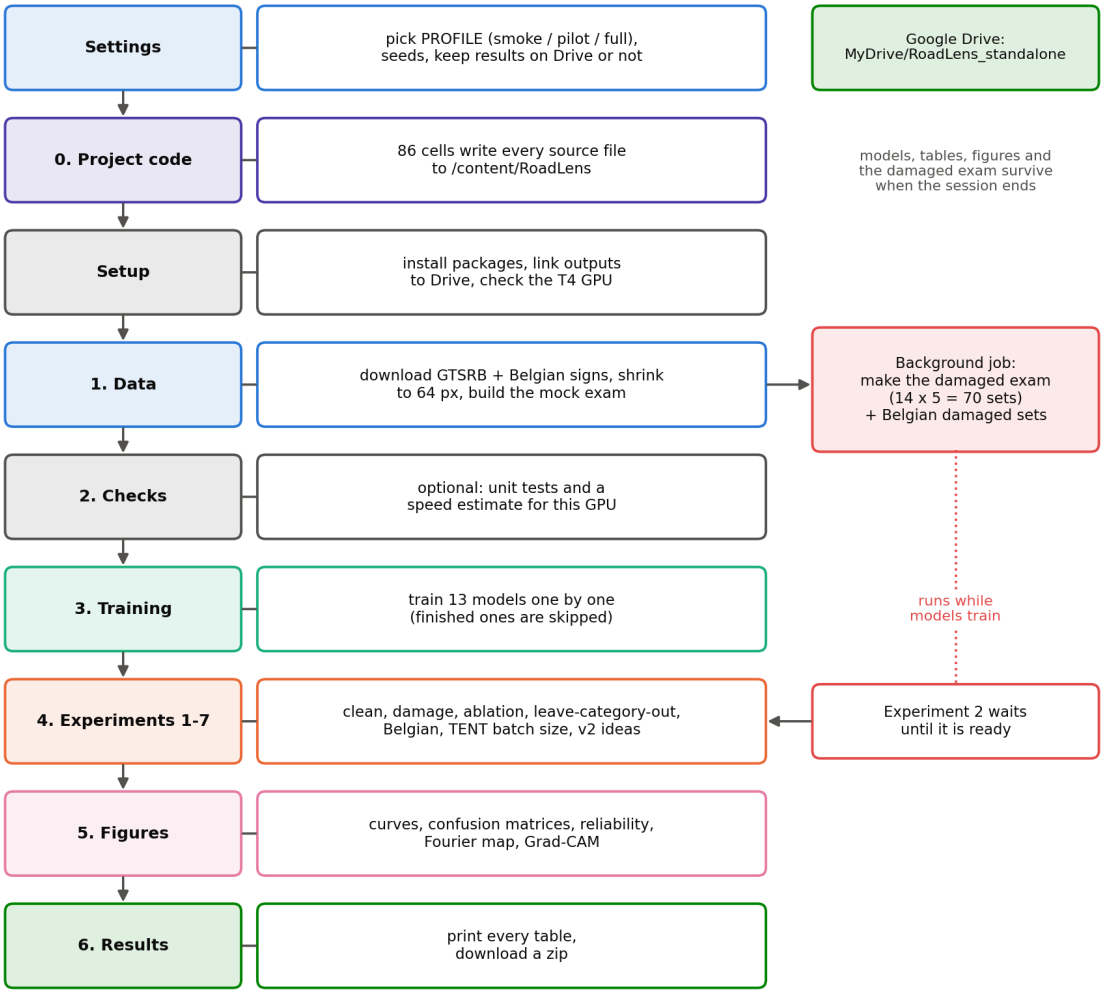
| Measure | Meaning | Good when |
|---|---|---|
| Accuracy | share of signs named correctly | high |
| Macro-F1 | accuracy that treats rare and common sign types equally | high |
| mCE | mistakes on damaged photos compared with SmallCNN (= 100) | below 100, the lower the better |
| ECE | is the model's confidence honest? (says 90% sure -> right 90% of the time) | close to 0 |

Every number is the average of 3 training runs with different random seeds, with the spread (mean +/- std), so a lucky run cannot fool you.

## The seven experiments

| # | Question it answers |
|---|---|
| 1 | How good is each model on clean photos? |
| 2 | How good is each model on every damage at every strength? (main table) |
| 3 | Which part of the method helps? Add one piece at a time: mixing, same-answer rule, extra changes, fusion, TENT. |
| 4 | If you tune the practice changes on noise + blur only, does it still help on weather + digital (and vice versa)? |
| 5 | Does a model trained on German signs read Belgian signs? |
| 6 | Does test-time adaptation still work when photos arrive 1, 16, 64 or 200 at a time? |
| 7 | The v2 ideas: self-normalised input, smoothed extra views, a smarter gate, honest confidence, safer adaptation. |

# The notebook: what happens when you press Run all

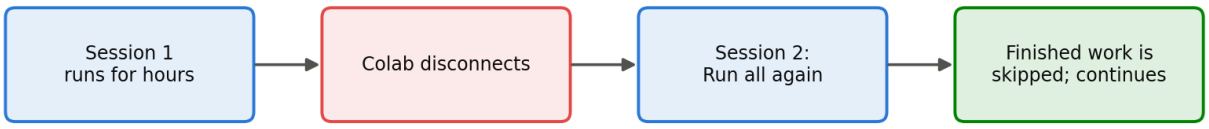



The notebook is one file that carries the whole project: no zip and no .py file is needed. The Belgian signs and GTSRB are downloaded from their official websites.

# The notebook, section by section

| Section | What it does | Time (rough, not measured) |
|---|---|---|
| Settings (first code cell) | PROFILE: 'smoke' = tiny check, 'pilot' = 1 seed and 2,000 exam photos, '' = full 3-seed run. KEEP_ON_DRIVE saves everything to MyDrive/RoadLens_standalone. RUN_CHECKS adds tests. | seconds |
| 0. Project code | 86 cells, one per source file (the roadlens package, scripts, settings files, tests). Each writes its file to /content/RoadLens. | seconds |
| Setup | Installs the needed Python packages, links the results folders to Drive, checks that a T4 GPU is attached, prepares helper functions. | 1-2 min |
| 1. Data | Downloads GTSRB (39k + 12.6k photos) and Belgian signs, shrinks them to 64 px, builds the mock exam. Starts making the damaged exam in the background (or restores it from Drive). Shows example damage pictures. | 5-10 min, then background |
| 2. Checks | Optional: unit tests and a speed estimate for this GPU. | ~5 min if on |
| 3. Training | Trains all 13 models one after another. Finished models are skipped on a re-run. | pilot ~1 h, full many hours |
| 4. Experiments 1-7 | Each experiment evaluates the models and writes tables (CSV, text, LaTeX). Experiment 2 first waits for the background damage job. | pilot ~20-30 min |
| 5. Figures | Accuracy-vs-strength curves, confusion matrices, reliability diagrams, Fourier heat map, Grad-CAM, as PDF, with previews. | ~10 min |
| 6. Results | Prints every table and downloads a zip of all results. | seconds |

## If Colab disconnects




Saved on Drive: finished models and every evaluation. A model that was cut off mid-training restarts from its first epoch.

# What you get at the end

| Folder (in MyDrive/RoadLens_standalone) | Contains |
|---|---|
| runs_pilot/ or runs/ | trained models, training logs, cached evaluations |
| results_pilot/ or results/exp1_clean ... exp7_improved | tables: .csv (numbers), .txt (readable), .tex (LaTeX for the paper) |
| results*/figures | PDF figures for the paper |
| results*/corruptions | pictures of every damage at every strength |
| corrupted_cache/ | the damaged exam, so the next session does not need to rebuild it |

## How to run it

• Open **RoadLens_standalone.ipynb** in Colab (upload it, or open it from Drive with Google Colaboratory).

• **Runtime > Change runtime type > T4 GPU.**

• **Runtime > Run all** and allow Google Drive access.

• Start with PROFILE = 'pilot'. When it finishes cleanly, set PROFILE = '' for the full three-seed run.

## Honest status

• A first pilot on Colab (10 Oct 2026, old milder damage) worked end to end: clean accuracy 98.8-99.6%, and the RoadLens training lifted ResNet-18 on damaged photos from 83.6% to 93.6%. A plain 50/50 average slightly beat the learned gate. These numbers are from one seed and no longer comparable, because the damage was made stronger.

• The standalone notebook, the stronger damage settings and the v2 ideas have not been run on a GPU yet. Your first run of the notebook is their first real test.

• Chinese signs (CTSD) are not included: the dataset needs a manual download and a checked class list.